

DevPilot (AI 研发领航) — 研发流程自动化解决方案

汇报完整版 · 每页 = 一页 PPT 的内容

第 1 页 · 封面

DevPilot (AI 研发领航)
研发流程自动化解决方案

汇报人: [姓名]

日期: 2026 年 6 月

版本: v1.0

一句话: 将"靠人记、靠人传、靠人写"的研发模式, 转变为"AI 自动执行 + 人做决策"的智能流水线。

第 2 页 · 我们面临的问题

研发团队每天都在被同一类问题消耗精力——

知识流失, 项目靠人传

代码仓库是完整的, 但"为什么当时这么设计""这个模块的边界在哪""改了 A 会不会炸 B"——这些信息存在人脑子里。人一走, 知识就丢了。新人接手一个 20 万行的项目, 通常需要 1-2 周才能开始有效贡献代码, 这期间全靠老人一对一口头传授。

文档和代码永远不同步

每个需求都要求写分析文档、PRD、设计文档、测试报告。但现实是: 需求赶、工期紧, 文档经常是"先写代码, 后补文档", 或者干脆不补。三个月后, 没人记得当初的设计决策。

变更影响全靠经验评估

客户说"加一个忘记密码功能", 到底要改几个模块? 改几行代码? 测试范围多大? 答案取决于"谁在评估"——老员工凭经验能说出个大概, 新人容易漏掉关键依赖。

流程割裂, 上下文反复切换

一个需求从提出到上线, 开发者要在"理解需求 → 分析 → 设计 → 编码 → 写测试 → 跑测试 → 写报告"七个阶段之间反复跳转。每个阶段使用的工具、关注的视角不一样, 大脑来回切换就是效率损失。

测试阶段被压缩

测试用例往往在编码完成后才补写, 而不是编码前就定义清楚。回归测试靠人工执行, 测试覆盖率难以量化。上线前的"最后一公里"最容易被忽视。

痛点	症状	本质
知识流失	文档过期，人员流动带走项目记忆	知识没有结构化沉淀
新人上手难	读代码 1-2 周才能贡献	缺少架构总览和模块字典
变更靠经验	影响范围评估因人而异	没有自动化的依赖分析
流程割裂	7 个阶段反复切换上下文	各环节工具链不打通
测试靠人	回归靠手动，覆盖率难量化	测试未自动化

第 3 页 · 我们的解决方案 — 双引擎架构（总览）

DevPilot 用两个核心引擎解决这些痛点：

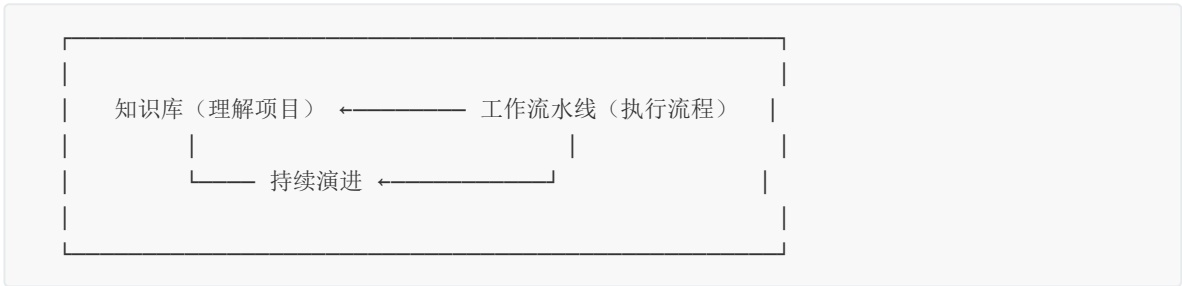
引擎一：知识库 — 将项目知识变成可检索、可继承的结构化资产

一键扫描整个项目，生成架构总览、模块字典、业务流程图、编码规范。后续每次需求完成后自动增量更新，不再依赖人的记忆。

引擎二：工作流水线 — 将研发流程从"人驱动"变为"AI 驱动 + 人决策"

从需求分析到测试报告的 7 个阶段全部由 AI 按规范执行，每个阶段必须人工确认才能进入下一步。AI 负责执行和产出，人负责审核和决策。

两者形成闭环：知识库让 AI 理解项目 → 流水线让 AI 走完流程 → 每次完成后再更新知识库 → 越用越准。



第 4 页 · 引擎一：知识库 — 沉淀项目资产

怎么用：新项目接入时，只需说一句"生成知识库"，AI 自动扫描整个项目代码。

产出物：

产出	内容	用途
架构总览	项目类型、技术栈、模块边界、目录结构	新人快速建立全局认知
模块字典	每个文件/模块做什么、依赖谁、被谁依赖	改代码前查影响范围
业务流程图 (PUML)	主流程、模块依赖图、协议路由、安全链路、OTA 状态机	理解业务逻辑，非开发人员也能看懂
编码规范	从现有代码中自动归纳的命名约定、代码模式、惯用法	保持新增代码风格一致
构建部署说明	编译脚本、打包流程、环境依赖	环境搭建不再靠口口相传

知识库不是一次性的：

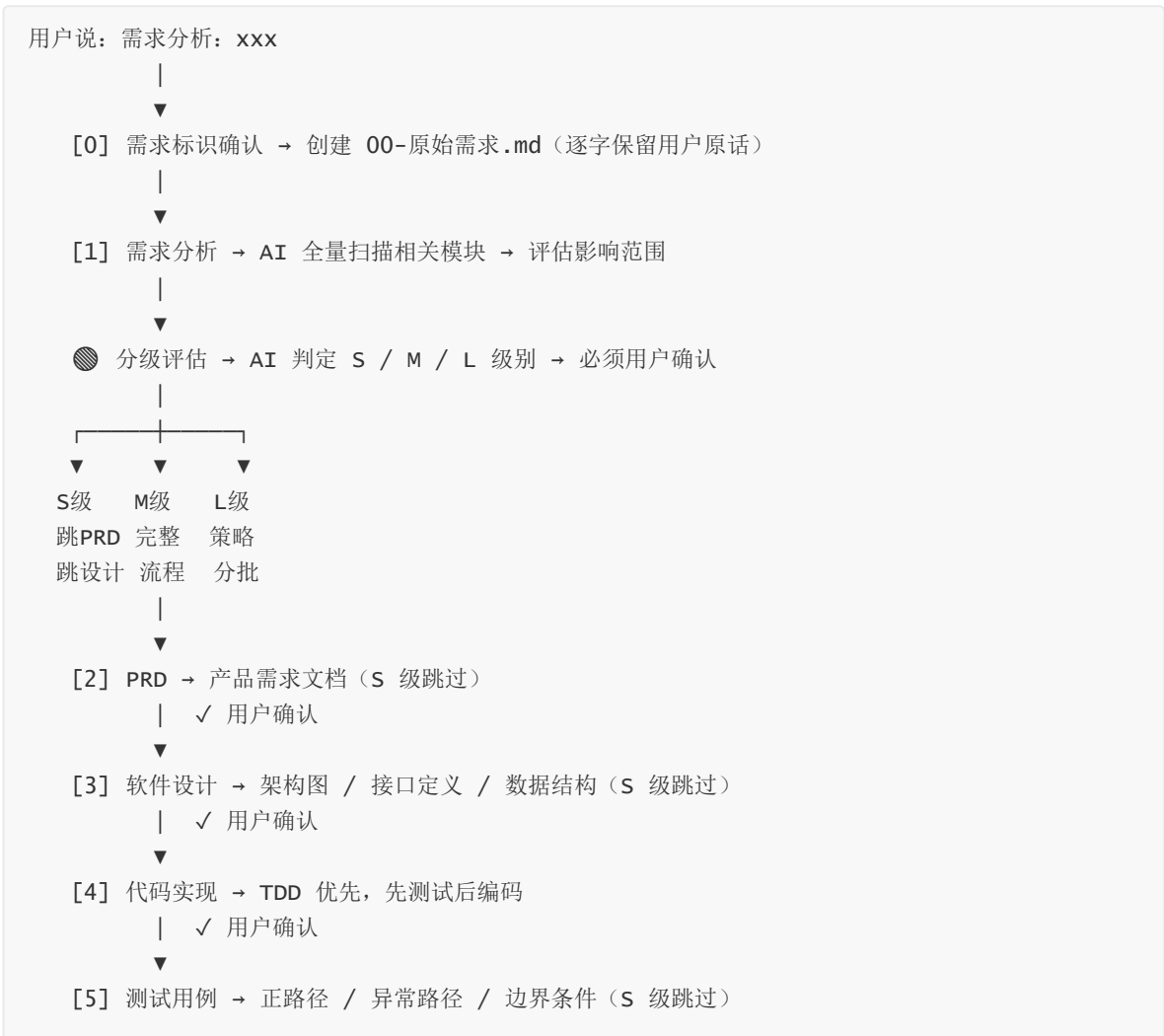
首次全量扫描（约 5 分钟）→ 后续每次需求完成 → 自动增量更新，只更新本次变更涉及的章节，不全量重扫（几秒完成）。

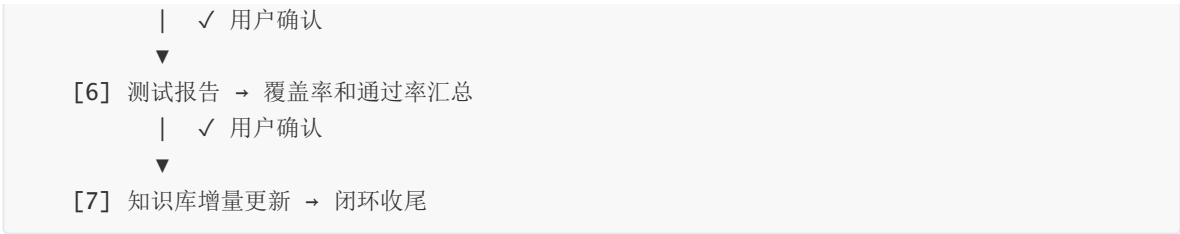
这意味着：项目持续迭代 → 知识库同步更新 → 团队始终有一份“活”的项目文档。即使核心开发人员离开，他的知识已经沉淀在知识库里。

第 5 页 · 引擎二：工作流水线 — 自动化研发流程

知识库让 AI 理解了项目，下一步是让 AI 按规范执行研发流程。

7 阶段全链路：





两个关键机制：

① 自动分级 (S / M / L)

不是所有需求都值得走全套流程，AI 自动评估规模并适配：

级别	判定标准	文件数	流程	典型场景
S	小改动	≤3	分析 → 直接改 → 报告 (跳过 PRD/设计/测试用例)	改个日志格式、修个边界条件
M	中等需求	4-15	完整 7 阶段 (分析→PRD→设计→TDD→测试→报告)	新增登录模块、改支付流程
L	大型需求	>15	策略文档 → 分批执行 → 回归清单	架构升级、多模块重构

级别由 AI 评估后必须用户确认。后续发现影响扩大时，只能升级不能降级。

② 门控确认

每个阶段完成后，AI 必须等用户确认才能进入下一阶段。你可以随时说"跳过设计直接编码""一次性生成分析+PRD"来调整节奏。AI 在轨道上运行，人掌握方向盘。

第 6 页 · 差异化：我们和已有工具的关系

团队已经在用 Claude Code，可能还装了 Superpowers 插件或 Copilot。DevPilot 和他们是什么关系？

vs Superpowers：不是替代，是分层

Superpowers 是一个研发工具箱——14 个独立技能（brainstorming、TDD、代码审查等），每个技能各自为战，需要人主动选择调用哪个。

DevPilot 是一套流水线——7 个阶段强制序列，门控规则自动注入每个会话，Superpowers 的技能只能在阶段内部被调用。

维度	Superpowers (工具箱)	DevPilot (流水线)
组织方式	独立技能，需要人主动选择	强制序列，自动匹配阶段
项目理解	每次从零开始读代码	知识库全流程共享上下文
变更尺度	不评估需求规模	自动分级 S/M/L
文档产出	自由路径，无标准命名	标准化 00-07 文件体系
可追溯性	没有原始需求记录	00-原始需求.md 逐字保留 + CHANGELOG
流程闭环	做完一个任务就结束	测试报告后自动更新知识库
规则加载	靠人记忆和习惯	SessionStart Hook 自动注入

打个比方：Superpowers 是一套质量很好的扳手和螺丝刀，DevPilot 是一条装配流水线——工具在流水线上用，但流水线管着工序和质检。

vs Copilot / Cursor：互补，不冲突

Copilot 和 Cursor 是代码补全工具，帮你在写代码的这一刻写得更快。

DevPilot 管的是写代码之前和之后：之前帮你做分析、PRD、设计；之后帮你做测试、报告、知识库更新。

三者可以同时用：DevPilot 管流程 → 代码实现阶段用 Copilot 提高编码速度 → Superpowers 的 TDD 技能保障代码质量。

第 7 页 · 技术架构（简要）

DevPilot 不是"自定义了几个 Claude Code Skill"那么简单，它是一套四层架构：

层	职责	核心文件
Hooks	会话启动时自动注入规则，任何人在框架目录下启动 Claude Code 都会生效	<code>load-behavioral-system.sh</code>
Rules	定义流程门控、触发关键字、输出路径、分级判定、文档模板	<code>CLAUDE.md</code> + <code>cicd-rules.md</code> + <code>DECISION.md</code>
Skills	6 个入口命令 + 自然语言触发（需求分析/PRD/设计/编码/测试）	<code>skills/jit-*</code>
Agent	15+ 专业化 Agent（component/feature/testing/quality/research...），流水线阶段的实际执行者	<code>.claude-collective/agents.md</code>

关键特性：规则通过 Hook 机制在会话启动时自动注入，不需要手动加载。优先级链是 `CLAUDE.md` > `外部技能` > `其他规则`，确保流水线门控不会被外部插件绕过。

第 8 页 · 落地验证：hw_swl_app 项目

我们在一个真实的工业级项目中验证了 DevPilot。

项目背景：

项	内容
项目	hw_swl_app — 无线安全通信网关
类型	嵌入式 PCIe 加密网卡的 OpenResty 网关服务
技术栈	Lua (46 模块) + C (5 个加密库) + Shell (运维脚本)
代码量	约 20 万行
复杂度	自定义 HWIOT 协议 (28 种消息类型)、国密 SM2/SM3/SM4、SKF 加密卡硬件交互、OTA 热更新

知识库产出：一句"生成知识库"，AI 在全量扫描后一次性生成：

- **PROJECT_KNOWLEDGE_BASE.md** (约 35KB)：架构总览、46 个模块的业务字典、技术栈和部署说明
- **PROJECT_KNOWLEDGE_DETAIL.md** (约 50KB)：深度分析，每个模块的接口定义和依赖链
- **8 张 PUBL 流程图**：主业务流程、模块依赖关系、HWIOT 协议路由、国密安全链路、SKF 硬件交互链、OTA 升级状态机、密钥层次体系、4 次握手时序
- **自动识别 8 大项目特征**：OpenResty 网关、自定义协议、国密加密、SKF 硬件交互、OTA 热更新、认证授权 等

这个知识库从 v1.0 迭代到 v6.5，经历了 6 次需求驱动下的增量更新，始终保持最新。

流水线产出：项目通过 DevPilot 执行了 3 个需求，每个需求都有完整的文档追溯链：

需求	级别	产出	特点
国际化技术底座	S	需求分析(22KB) + 测试报告 + CHANGELOG	AI 发现需求误判：原以为要建语言包系统，实际只需改 4 个文件
随机MAC接口参数修改	S	密钥体系分析 + 8/8 测试通过 + 国密/国际双分支	精准改动 1 个文件，Body 22B→54B
项目知识库	—	8 文档 + 8 流程图 + 6 次迭代	持续演进，知识沉淀

效率对比：

维度	传统方式	DevPilot 方式
新人理解项目	1-2 周读代码 + 问人	读知识库，半天建立全局认知
需求分析	人工扫描代码，半天到一天	AI 自动扫描，10-30 分钟出初稿
文档完整性	常缺失或过期	每个需求 00-06 标准化输出
测试覆盖	靠人记得测什么	AI 自动覆盖正常/异常/边界路径

第 9 页 · 现存挑战

我们需要诚实地面对当前版本的局限性。这体现了我们对产品的深度认知和改进意愿。

挑战一：未对接外部知识源——AI 有时在"猜"

当前 AI 做需求分析和设计时，数据来源是：模型训练数据 + 当前会话喂的上下文 + 项目知识库。它不知道 Confluence 上有一篇架构决策记录，不知道数据库的这张表已经在另一个项目里改了字段类型，不知道 Jira 上有个相关 issue 在上周刚刚被标记为 blocked。

后果：AI 在缺乏外部信息时可能做出错误假设，需要人工审核来纠正。

挑战二：测试闭环尚未完全自动化

当前流水线可以产出测试用例文档和测试报告，但测试的执行和验证这一环还需要开发人员手动触发——linter 报错、编译失败、单元测试失败，这些应该自动跑、自动修。

挑战三：通用模型面对特定领域有时犯错

面对极其复杂的自定义协议（如 HWIOT 的 28 种消息类型）、特定硬件的寄存器操作序列、国密算法的某些边界条件，通用大模型可能反复在同一个坑里犯错。这不是模型能力不够，而是它没见过你的领域。

第 10 页 · 未来路线图

针对这三个挑战，我们制定了清晰的改进路线。



P0 · 本月启动（改动最小，收益最直接）

方向	做什么	预期价值
RAG 第一步	流水线阶段自动检索已有知识库，分析时标注引用来源	"根据 PROJECT_KNOWLEDGE_BASE.md §2.3, 该模块..."——消除凭空猜测
静态分析闭环	代码生成后自动跑 linter/编译器，错误喂回 AI 自动修正	秒级发现语法错误和类型不匹配，无需人工 review
自动化测试基础	测试用例同时输出可执行脚本（LuaUnit/curl），测试报告阶段自动跑	从"写了用例"进化到"跑了验证"，测试真正自动化

P1 · Q3（打通外部系统，形成完整工具链）

方向	做什么	预期价值
RAG 第二步	对接 Confluence/Wiki API → 检索历史 PRD 和架构决策；对接数据库 Schema → 理解表结构	AI 分析需求时自动检索"有没有类似需求做过""数据库里这个字段是什么类型"
架构图交互	PUML → Mermaid + click，模块节点可跳转源码、看 commit 历史、关联需求标识	架构图从静态图片变成活的系统地图
Jira 双向集成	流水线状态自动同步 Jira（创建 Story / 更新状态 / 追加 comment）	PM 在 Jira 上实时看到研发进度，不再断层
Web/UI 测试	集成 Playwright，前端项目自动跑页面回归测试	前端改动后自动截图对比基线，发现 UI 异常

P2 · Q3-Q4（战略储备）

方向	做什么
微调数据收集	每次 AI 犯错修正的 case 标记为 [微调素材]，积累需求→正确实现的映射对子
内部插件市场	团队可以发布和共享自定义 Agent/Skill，形成内部生态

P3 · 2027（长期目标）

当微调数据集超过 500 个高质量样本时，启动模型微调实验。让 AI 从"懂通用编程"进化为"懂你的业务语言和硬件细节"。

第 11 页 · 总结

DevPilot 为团队带来的核心价值：

1. 项目知识不再随人走

知识库把"靠人传"变成"结构化沉淀"，新人半天就能建立全局认知。

2. 研发流程有规范可循

从需求到报告的 7 个阶段，AI 自动执行，人工确认把关。每个阶段的产出都标准化、可追溯。

3. 已在真实项目中验证

在 20 万行的工业级嵌入式项目中跑通了完整闭环，知识库迭代 6 个版本，3 个需求完成全流程交付。

4. 有清晰的进化路线

P0 本月见效，P1 打通外部系统，P2-P3 战略储备。我们清楚现在的不足，也知道下一步怎么走。

第 12 页 · 行动建议

优先级	行动	预期效果	时间
1	在 1-2 个新项目中试点推广	验证通用性，收集不同场景的反馈	立即
2	启动 P0 改进（RAG + 静态分析 + 自动化测试）	消除核心短板，提升 AI 输出可信度	本月
3	成立 2-3 人维护小组	跟进 P1 改进，响应团队内部定制需求	Q3
4	每季度输出使用报告	量化效率提升，为推广决策提供数据	持续

第 13 页 · 谢谢

DevPilot · AI 研发领航

将"靠人记、靠人传、靠人写"→"AI 自动执行 + 人做决策"

详细技术文档见：[DevPilot产品介绍.md](#)

下篇：研发人员使用指南

安装与启动

前置要求： Node.js >= 18 + git + Claude Code CLI

```
git clone <仓库地址> claude-code-autopilot
cd claude-code-autopilot
bash install.sh --tools
```

启动： 在项目目录下启动 Claude Code，输入 `/jit-devpilot-init` 即可激活。框架自动识别当前目录为目标项目，不需要手动指定路径。

触发方式速查

你想做什么	这样说
让 AI 理解项目（第一步）	生成知识库
开始新需求	需求分析：功能描述
生成 PRD	生成PRD
软件设计	软件设计 或 变更策略
写代码	代码实现
测试用例	测试用例 或 回归验证
测试报告	测试报告
修改已有需求	需求变更：user-login 增加忘记密码
同步知识库	知识库更新
查看项目进度	/jit-project-autopilot-status

目标项目路径只需指定一次，后续自动继承。在项目目录下启动时自动识别。

可用 Skill 命令： /jit-devpilot-init /jit-project-knowledge-base /jit-project-autopilot-status /jit-env-auto-setup /jit-ui-ux-pro-max

典型 workflow

 生成知识库

 扫描完成，知识库已生成

 需求分析：用户登录模块

 需求标识建议：user-login ->  M 级（约 8 文件，2 模块）

 确认

 [创建 00-原始需求.md + CHANGELOG.md] 需求分析完成，确认后继续 PRD？

 生成PRD

 PRD 完成

 软件设计

 设计完成

 开始编码

 代码+测试完成

 测试报告

 12/12 通过 -> 知识库已增量更新 ☒

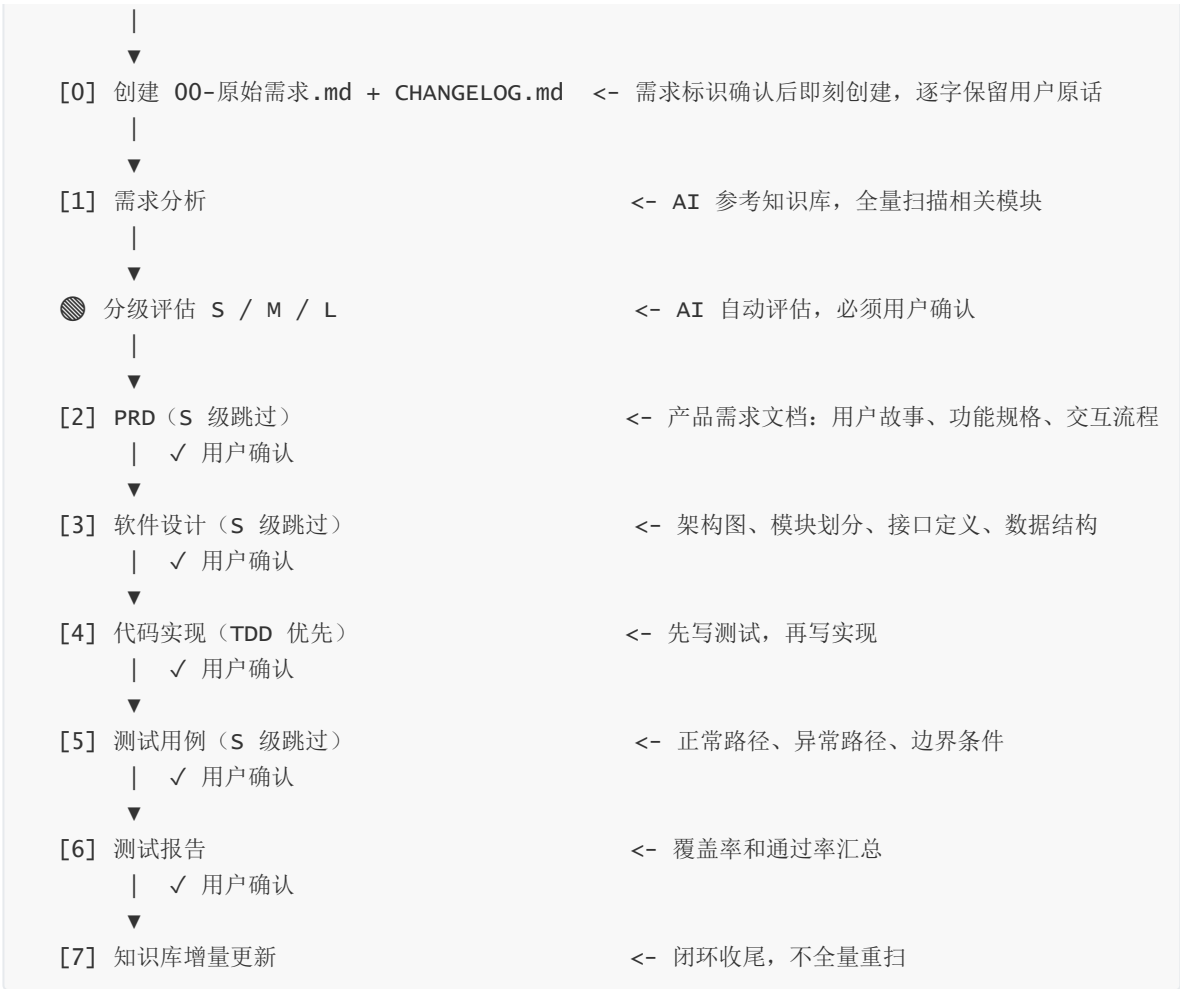
第一步：让 AI 理解项目

每个阶段 AI 完成后等你确认。可以说"跳过设计直接编码"或"一次性生成分析+PRD，我一起看"来调整节奏。S 级自动跳过 PRD/设计/测试用例。

流水线全流程详解

场景一：正常需求实现

用户自然语言触发



场景二：需求变更



自动分级 (S / M / L)

级别	判定	文件数	流程差异
S	小改动	<=3	跳 PRD、设计、测试用例。简要分析 -> 直接改 -> 简要报告
M	中等需求	4-15 或 2-3 模块	完整 7 阶段：分析->PRD->设计->TDD->测试用例->测试报告
L	大型需求	>15 或跨模块	策略文档（change-strategy）-> 分批执行 -> 回归清单

级别由 AI 评估后必须用户确认，只能升级不能降级。

技术架构详解

DevPilot 不是"自定义了几个 Claude Code Skill"，而是一套完整的四层架构：

Layer 1 - Hooks（会话注入层）

5 个 Hook 脚本，最重要的是 `load-behavioral-system.sh`——SessionStart 触发，每次启动 Claude Code 时自动将 `DECISION.md` + `cicd-rules.md` 注入当前会话。任何人在框架目录下启动，规则自动生效。

Hook	触发时机	作用
<code>load-behavioral-system.sh</code>	SessionStart	注入 <code>DECISION.md</code> + <code>cicd-rules.md</code>
<code>directive-enforcer.sh</code>	每次对话	流程门控强制执行
<code>block-destructive-commands.sh</code>	命令执行前	拦截危险操作
<code>collective-metrics.sh</code>	任务完成	指标采集
<code>test-driven-handoff.sh</code>	阶段切换	TDD 纪律检查

Layer 2 - Rules（规则层）

文件	优先级	作用
<code>CLAUDE.md</code>	最高	流程门控不可绕过、触发关键字表、分级规则、质量要求
<code>cicd-rules.md</code>	会话注入	输出路径规则、目标项目自动识别、00-原始需求模板、分级判定表
<code>DECISION.md</code>	会话注入	自动委派决策引擎（什么情况走什么 Agent）
<code>README.md</code>	事实源	完整流程说明，跨文件一致性锚点

Layer 3 - Skills (用户入口层)

6 个 jit-* 前缀 Skill:

Skill	命令	用途
jit-devpilot-init	/jit-devpilot-init	激活流水线，自动识别目标项目
jit-project-knowledge-base	/jit-project-knowledge-base	全量扫描生成项目知识库
jit-project-autopilot-status	/jit-project-autopilot-status	查看项目各需求进度
jit-env-auto-setup	/jit-env-auto-setup	环境自动检测与配置
jit-ui-ux-pro-max	/jit-ui-ux-pro-max	UI/UX 智能设计引擎
jit-nowTimeAndModel	/jit-nowTimeAndModel	时间模型工具

流水线阶段（需求分析/PRD/设计/编码/测试） **不通过 Skill 触发**，而是自然语言关键字匹配 -> /van 命令 -> Agent 执行。

Layer 4 - Agent (执行层)

/van 命令读取 README.md 确定当前阶段的执行规范，从 agents.md 的 15+ 专业化 Agent 中选择合适的分派任务：

Agent	职责	适用阶段
@component-implementation-agent	UI 组件 + TDD	代码实现
@feature-implementation-agent	业务逻辑 + TDD	代码实现
@infrastructure-implementation-agent	构建系统 + TDD	代码实现
@testing-implementation-agent	测试套件	测试用例
@polish-implementation-agent	性能优化	代码实现
@quality-agent	代码审查 + 安全分析	测试报告
@devops-agent	部署 + CI/CD	构建部署
@functional-testing-agent	Playwright 浏览器测试	测试用例
@research-agent	Context7 技术调研	需求分析/设计
@prd-research-agent	PRD -> 任务生成	PRD
@task-orchestrator	任务协调并行	全流程

与 Superpowers 的详细对比

Superpowers 有哪些技能

Superpowers 是一个研发工具箱，提供 14 个独立技能：

技能	功能
brainstorming	需求探讨 -> 设计方案 -> 写 spec -> 用户审核
writing-plans	将 spec 拆成 bite-sized 实现计划
subagent-driven-development	每任务分派独立 subagent，两阶段 review
test-driven-development	强制 TDD：不写测试不许写实现
executing-plans	并行 session 执行计划
dispatching-parallel-agents	并行分派多个 agent
requesting-code-review	发起代码审查
receiving-code-review	处理代码审查反馈
systematic-debugging	系统化调试方法
finishing-a-development-branch	分支收尾 + PR
verification-before-completion	完成前自验证
using-git-worktrees	Git worktree 隔离环境
writing-skills	编写自定义 Skill

关键区别

维度	Superpowers (工具箱)	DevPilot (流水线)
组织方式	14 个独立技能，各自为战	7 阶段强制序列，门控不可绕过
项目理解	无，每次从零开始读代码	知识库全流程共享上下文
变更尺度	无，不评估需求规模	自动分级 S/M/L，按级适配流程
文档产出	docs/superpowers/specs/ 自由路径	docs/{需求标识}/ 标准化 00-07 命名
可追溯性	无，不知道需求从哪来	00-原始需求.md 逐字保留 + CHANGELOG.md
流程闭环	无，做完任务就结束	测试报告 -> 知识库增量更新
规则注入	无，靠人记住调用哪个技能	SessionStart Hook 自动注入

两者的关系：分层协作



FLOW GATE 规则明确规定：Superpowers 只能在流水线阶段内部被调用，绝不允许替代或绕过流水线入口。即使 brainstorming 和"需求分析"匹配度 100%，也必须先走 DevPilot 门控。

知识库详解

知识库是 DevPilot 的根基

新人接手项目，第一件事是"读代码、看文档、理解架构"——通常 1-2 周。DevPilot 一句"生成知识库"，AI 自动扫描整个项目：

产出	说明
架构总览	项目类型、技术栈、模块划分、目录结构
模块字典	每个模块做什么、依赖谁、被谁依赖
流程图（PUML）	主业务流程、模块调用链、协议路由、安全链路、OTA 状态机
编码规范	从代码中自动归纳的命名约定、代码模式、惯用法
构建部署说明	编译脚本、打包流程、环境依赖

一次生成，全流程复用。后续所有需求分析、代码实现、测试都会自动参考知识库。

知识库持续演进

首次全量扫描（约 5 分钟）-> 需求 A 完成 -> 增量更新（几秒）-> 需求 B 完成 -> 增量更新（几秒）-> ... 越用越准。

每次只更新本次变更涉及的章节，不全量重扫。项目知识持续沉淀，不随人员离职而丢失。

需求目录结构

```
目标项目/
├── docs/
│   ├── knowledge-base/                # 项目知识库（全局共享，首次全量 -> 后续增量）
│   │   ├── PROJECT_KNOWLEDGE_BASE.md
│   │   ├── PROJECT_KNOWLEDGE_DETAIL.md
│   │   └── *.puml
│   └──
└── user-login/                        # 需求 A（独立目录，互不干扰）
```

			00-原始需求.md	# 逐字保留用户原话
			01-requirements-analysis.md	
			02-prd.md	
			03-software-design.md	
			04-test-cases.md	
			05-test-report.md	
			CHANGELOG.md	# 全程变更记录
			payment-module/	# 需求 B
			tests/{需求标识}/	# 测试代码

团队定制化拓展

DevPilot 不是黑盒。团队可根据技术栈和流程规范定制：

定制需求	改哪里	难度
调整触发关键字	CLAUDE.md 触发规则表	★
修改 S/M/L 分级阈值	cicd-rules.md 分级判定表	★
自定义文档模板	cicd-rules.md 00-原始需求模板 / CHANGELOG 模板	★
添加项目专属 Skill	skills/ 目录新增 SKILL.md	★★
调整 Agent 分派策略	.claude-collective/agents.md	★★
添加自定义 Hook	.claude/hooks/ 新增脚本	★★★
修改流程阶段顺序	CLAUDE.md + README.md + cicd-rules.md 三处同步	★★★
扩展知识库扫描策略	skills/jit-project-knowledge-base/SKILL.md	★★★

典型定制场景

场景一：团队用 Jira 管理需求

在 cicd-rules.md 追加 Jira 触发规则，需求分析时自动关联 Issue ID，PRD 输出到 Jira 描述字段。

场景二：Go 微服务专属 Agent

在 agents.md 新增 @go-microservice-agent，注入 Go 规范 + gRPC 最佳实践。代码实现阶段自动选择。

场景三：金融合规强制审查门

在 .claude/hooks/ 新增 compliance-gate.sh，代码完成后强制安全扫描，不通过则阻断。

场景四：设计系统定制

修改 jit-ui-ux-pro-max Skill 的设计规范文件，注入团队设计系统。

核心文件速查


```
claude-code-autopilot/
├─ CLAUDE.md # 根规则：改触发关键字、流程定义
├─ .claude-collective/
│   └─ cicd-rules.md # 硬规则：改分级阈值、输出模板、路径规则
│   └─ DECISION.md # 委派引擎：改自动决策逻辑
│   └─ agents.md # Agent 定义：加领域专属 Agent
├─ .claude/hooks/ # Hook 脚本：加自定义阶段门控
├─ skills/ # 入口 Skill：加项目专属命令
└─ README.md # 流程事实源：改流程时必同步
```

常见问题

Q: 知识库每次都要重建吗？

A: 不用。首次全量扫描（约 5 分钟），后续"知识库更新"只增量更新变更部分（几秒）。

Q: 必须按顺序走完所有阶段吗？

A: 不一定。可以说"跳过设计直接编码"。S 级自动跳过 PRD/设计/测试用例。

Q: 中途需求变了怎么办？

A: 说"需求变更：xxx"，AI 自动分析影响范围，只重做受影响的部分。

Q: 新开会话怎么恢复？

A: "/jit-project-autopilot-status"，AI 自动扫描已有产出，告诉你进度和下一步。

Q: DevPilot 和 Superpowers 冲突吗？

A: 不冲突。DevPilot 管流程（什么阶段做什么），Superpowers 管执行（具体怎么做）。阶段内部可调用。

Q: 和 Copilot / Cursor 有什么区别？

A: Copilot/Cursor 是代码补全工具，DevPilot 是研发流程管理工具——编码前做分析/设计，编码后做测试/报告/知识库。互补不冲突。

Q: 目标项目路径每次都要指定吗？

A: 不需要。首次指定后自动继承。在项目目录下启动 claude 时自动识别当前目录。

Q: 怎么跳过不需要的阶段？

A: 说"跳过设计直接编码"即可。S 级需求自动跳过 PRD、设计和测试用例阶段。

完整产品介绍见：[DevPilot产品介绍.md](#) | 快速入门：[快速入门.md](#) | 升级指南：[升级指南.md](#)